

计算机系统结构

第8章 I/O-外存系统

曹强

caoqiang@hust.edu.cn

武汉光电国家研究中心
计算机学院存储所



- 8. 1 [I/O系统的性能](#)
- 8. 2 [I/O系统的可靠性、可用性和可信性](#)
- 8. 3 [廉价磁盘冗余阵列RAID](#)
- 8. 4 [总线](#)
- 8. 5 [通道处理机](#)
- 8. 6 [I/O与操作系统](#)

8.1 I/O系统的性能

1. 输入/输出系统简称I/O系统

包括：

- I/O设备
- I/O设备与处理机的连接

2. I/O系统是计算机系统中的一个重要组成部分

- ✓ 完成计算机与外界的信息交换
- ✓ 给计算机提供大容量的外部存储器

3. 按照主要完成的工作进行分类：

- 存储I/O系统（本章内容）
- 通信I/O系统

8.1 I/O系统的性能

4. I/O系统的性能对CPU的性能有很大的影响，若两者的性能不匹配，I/O系统就有可能成为整个系统的瓶颈。
5. 系统的响应时间(衡量计算机系统的一个更好的指标)
 - ✓ 从用户输入命令开始，到得到结果所花费的时间。
 - ✓ 由两部分构成：
 - I/O系统的响应时间
 - CPU的处理时间
6. 多进程技术只能够提高系统吞吐率，并不能够减少系统响应时间。

8.1 I/O系统的性能

7. 评价I/O系统性能的参数主要有：

- ✓ 连接特性

（哪些I/O设备可以和计算机系统相连接）

- ✓ I/O系统的容量

（I/O系统可以容纳的I/O设备数）

- ✓ 响应时间和吞吐率等

8. 另一种衡量I/O系统性能的方法：

考虑I/O操作对CPU的打扰情况。

即考查某个进程在执行时，由于其他进程的I/O操作，使得该进程的执行时间增加了多少。

8.2 I/O系统的可靠性、可用性和可信性

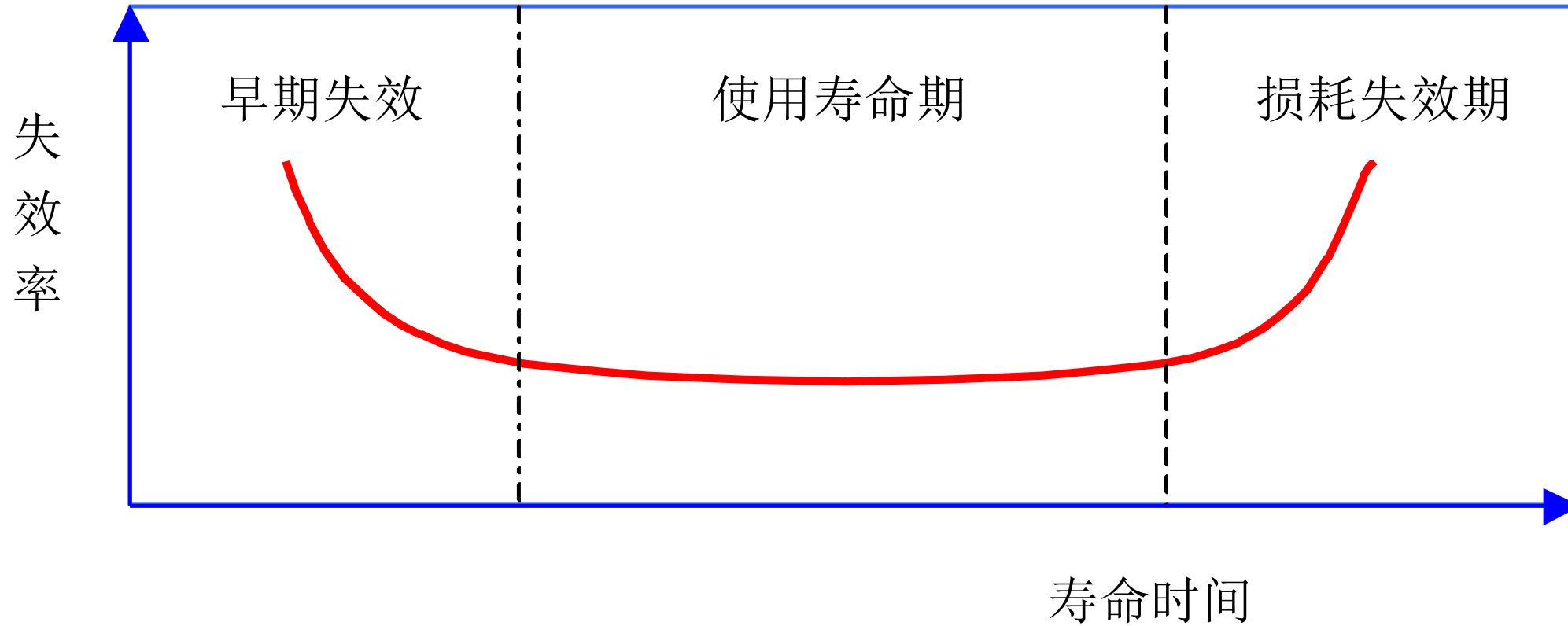
1. 反映外设可靠性能的参数有：

- ✓ 可靠性 (Reliability)
- ✓ 可用性 (Availability)
- ✓ 可信性 (Dependability)

2. 系统的可靠性：系统从某个初始参考点开始一直连续提供服务的能力。

- ✓ 用平均无故障时间MTTF来衡量。
- ✓ 系统中断服务的时间用平均修复时间MTTR来衡量。

8.2 I/O系统的可靠性、可用性和可信性-产品的寿命特性



8.2 I/O系统的可靠性、可用性和可信性

- ✓ MTTF的倒数就是系统的失效率。
- ✓ 如果系统中每个模块的生存期服从指数分布，则系统整体的失效率是各部件的失效率之和。

3. 系统的可用性：系统正常工作的时间在连续两次正常服务间隔时间中所占的比率。

$$\text{可用性} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

- ✓ MTTF+MTTR：平均失效间隔时间MTBF

(Mean Time Between Failure)

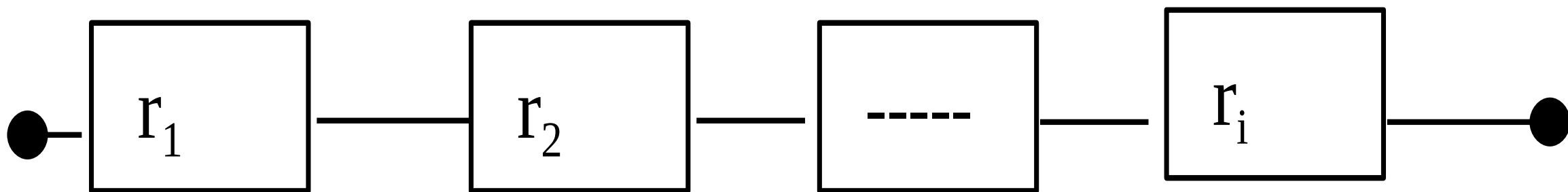
4. 系统的可信性：服务的质量。即在多大程度上可以合理地认为服务是可靠的。（不可以度量）

8.2 I/O系统的可靠性、可用性和可信性-MTBF和MTTF

1. 对不可维修的产品的平均寿命是指从开始投入工作，至产品失效的时间平均值。也称平均失效前时间，记以MTTF，它是英文（Mean Time To Failure）的缩写。
2. 对可维修产品而言，其平均寿命是指两次故障间的时间平均值，称平均故障间隔时间，习惯称平均无故障工作时间，用MTBF记之，它是英文（Mean Time Between Failures）的缩写。

8.2 I/O系统的可靠性、可用性和可信性-串联系统

1. 定义：系统中的下属几个组件全部工作正常时，系统才正常；当系统中有一个或一个以上的组件失效时，系统就失效，这样的系统就称串联系统。串联系统的可靠性框图，就是下属几个组件的串联图。 设系统下属组件的可靠度分别为
2. $r_1, r_2, \dots, r_n$
3. 串联系统的框图为



8.2 I/O系统的可靠性、可用性和可信性-串联系统

4. 用 S_s 和 S_i 分别表示系统和单元的正常工作状态，则依据串联系统的定义，串联系统中正常事件是“交”的关系，逻辑上为“与”的关系，系统要正常工作，必须各子系统都正常工作，则有

$$S_s = S_1 \cap S_2 \cap S_3 \cap \cdots \cap S_n$$

8.2 I/O系统的可靠性、可用性和可信性-串联系统

系统正常工作的概率为各单元概率之积，因此

$$P_s \{S_s\} = \prod_{i=1}^n P_i \{S_i\}$$

由于

$$P_s = R_s(t) \quad P_i(S_i) = r_i(t)$$

所以

$$R_s = r_1 \bullet r_2 \cdots r_n = \prod_{i=1}^n r_i$$

对于指数分布

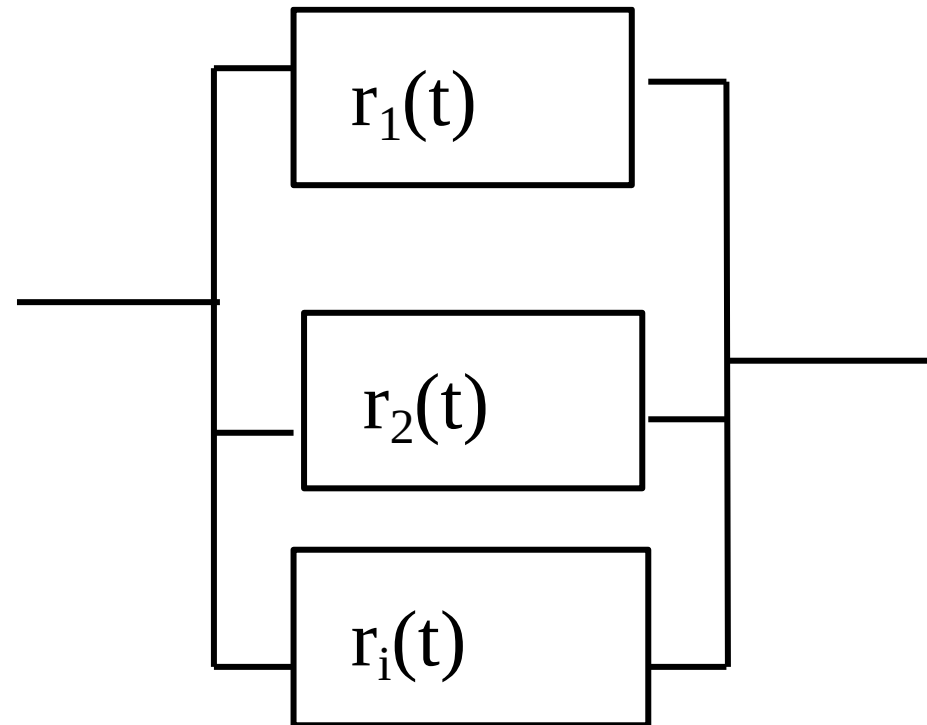
$$r_i(t) = e^{-\lambda_i t} \quad \therefore r(t) = e^{-\left(\sum_{i=1}^n \lambda_i\right) t}$$

$$\lambda = \sum_{i=1}^n \lambda_i \quad MTBF = \frac{1}{\sum_{i=1}^n \lambda_i}$$

8.2 I/O系统的可靠性、可用性和可信性-并联系统

定义：系统中的几个下属组件，只要其中一个工作正常，则系统就正常工作，只有全部组件都失效时，系统才失效，这样的系统就称并联系统。并联系统的可靠性方框图为n个组件的并联图。

1. 设组成组件的可靠度分别为 $r_1, r_2, \dots, r_n$
2. 相应组件的失效（故障）概率分别为 $q_1, q_2, \dots, q_n$
3. 并设并联系统的失效（故障）概率为 Q_s



8.2 I/O系统的可靠性、可用性和可信性-并联系统

1. 用 S_s 和 S_i 分别表示系统和单元的正常工作状态，用 F_s 和 F_i 表示系统和单元不正常工作，则依据并联系统的定义，并联系统中不正常事件是“交”的关系，逻辑上为“与”的关系，系统要不正常工作，必须各子系统都不正常工作，则有

$$F_s = F_1 \cap F_2 \cap F_3 \cap \cdots \cap F_n$$

8.2 I/O系统的可靠性、可用性和可信性-并联系统

系统不正常工作的概率为各单元不正常工作概率之积，因此

$$F_s \{S_s\} = \prod_{i=1}^n F_i \{S_i\}$$

$$\text{由于 } F_s = Q_s(t) \quad F_i(S_i) = q_i(t)$$

$$\text{所以 } Q_s = q_1 \cdot q_2 \cdots q_n = \prod_{i=1}^n (1 - r_i)$$

$$R_s = 1 - Q_s = 1 - \prod_{i=1}^n q_i = 1 - \prod_{i=1}^n (1 - r_i)$$

8.2 I/O系统的可靠性、可用性和可信性-并联系统

对于指数分布,若失效率用 λ 表示

$$MTBF = \int_0^{\infty} R_s(t) dt$$

$$MTBF = \sum_{i=1}^n \frac{1}{\lambda_i} - \sum_{1 \leq i < j \leq n} \frac{1}{\lambda_i + \lambda_j} + \cdots + (-1)^{n-1} \frac{1}{\lambda_1 + \lambda_2 + \cdots + \lambda_n}$$

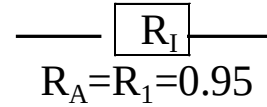
当N个相同时, 则

$$= \int_0^{\infty} \{1 - (1 - e^{-\lambda t})^n\} dt$$

$$= \frac{1}{\lambda} + \frac{1}{2\lambda} + \cdots + \frac{1}{n\lambda}$$

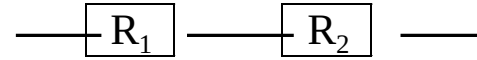
8.2 I/O系统的可靠性、可用性和可信性-串联模型计算示例

任务 A:



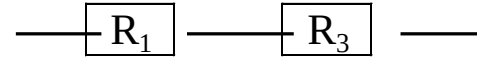
$$R_A = R_1 = 0.95$$

任务 B:



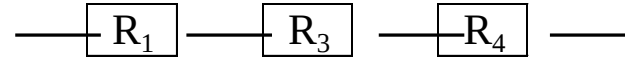
$$R_B = R_1 R_2 = (0.95)(0.93) = 0.88$$

任务 C:



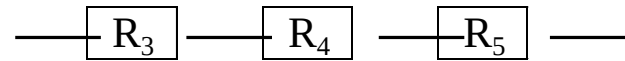
$$R_C = R_1 R_3 = (0.95)(0.99) = 0.94$$

任务 D:



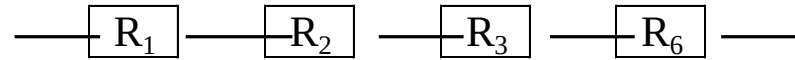
$$R_D = R_1 R_3 R_4 = (0.95)(0.99)(0.91) = 0.85$$

任务 E:



$$R_E = R_3 R_4 R_5 = (0.99)(0.91)(0.90) = 0.81$$

任务 F:

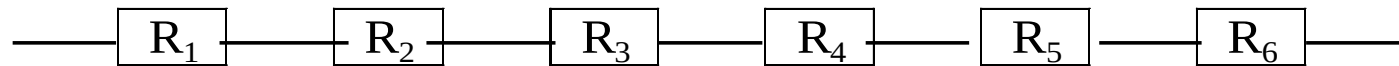


$$\begin{aligned} R_F &= R_1 R_2 R_3 R_6 \\ &= (0.95)(0.93)(0.99)(0.95) \\ &= 0.83 \end{aligned}$$

8.2 I/O系统的可靠性、可用性和可信性-串联模型计算示例

① 在 3h 中成功完成全部 6 项任务的概率 P_s 等于 6 个设备的可靠度之积。因为为了能在 3h 中成功地完成全部 6 项任务，所有设备必须工作。

②



$$\begin{aligned} P_s &= R_1 R_2 R_3 R_4 R_5 R_6 \\ &= (0.95) (0.93) (0.99) (0.91) (0.90) (0.95) \\ &= 0.68 \end{aligned}$$

值得注意的是，成功完成6项任务的概率 P_s 不等于完成各项任务可靠度 R_A 、 R_B 、 R_C 、 R_D 、 R_E 、 R_F 的乘积。因为有的设备，如设备1、设备2、设备3及设备4具有多功能。若采用这种任务可靠度相乘的办法，将会使某些设备多次参加计算，从而造成错误计算。这是一个典型的多功能部件的例子,后面还会讲到。

8.2 I/O系统的可靠性、可用性和可信性-混联系统的可靠性公式

可靠度： $R(t)$ ，不可靠度： $F(t)$ ，关系式： $R(t) + F(t) = 1$ 。

在各单元相同情况下，用 n 代表串联倍数， m 代表并联倍数。

①串并联系统：先串联、后并联

$$\text{可靠度 } R(t) = 1 - [1 - R_i^n(t)]^m。$$

②并串联系统：先并联、后串联

$$\text{可靠度 } R(t) = [1 - (1 - R_i(t))^m]^n。$$

公式用途：RAID 0+1是串并联系统，RAID 1+0是并串联系统

8.2 I/O系统的可靠性、可用性和可信性-例8.1

例8.1 假设磁盘子系统的组成部件和它们的MTTF如下：

- (1) 磁盘子系统由10个磁盘构成，每个磁盘的MTTF为1000000小时；
- (2) 1个SCSI控制器，其MTTF为500000小时；
- (3) 1个不间断电源，其MTTF为200000小时；
- (4) 1个风扇，其MTTF为200000小时；
- (5) 1根SCSI连线，其MTTF为1000000小时。

假定每个部件的生存期服从指数分布，同时假定各部件的故障是相互独立的，求整个系统的MTTF。

8.2 I/O系统的可靠性、可用性和可信性-例8.1

解 整个系统的失效率为：

$$\text{系统失效率} = 10 \times \frac{1}{1000000} + \frac{1}{500000} + \frac{1}{200000} + \frac{1}{200000} + \frac{1}{1000000} = \frac{23}{1000000}$$

系统的MTTF为系统失效率的倒数，即：

$$\text{MTTF} = \frac{1000000}{23} = 43500 \text{小时}$$

即将近5年。

8.2 I/O系统的可靠性、可用性和可信性

5. 提高系统组成部件可靠性的方法

- ✓ 有效构建方法 (valid construction)

在构建系统的过程中消除故障隐患，这样建立起来的系统就不会出现故障。

- ✓ 纠错方法 (error correction)

在系统构建中采用容错的方法。这样即使出现故障，也可以通过容错信息保证系统正常工作。

8.3 廉价磁盘冗余阵列RAID

1. **磁盘阵列DA (Disk Array)**：使用多个磁盘（包括驱动器）的组合来代替一个大容量的磁盘。
 - ✓ 多个磁盘并行工作。
 - ✓ 以条带为单位把数据均匀地分布到多个磁盘上。
(交叉存放)
 - ✓ 条带存放可以使多个数据读/写请求并行地被处理，从而提高总的I/O性能。
2. 这里并行性有两方面的含义：

8.3 廉价磁盘冗余阵列RAID

✓多个独立的请求可以由多个盘来并行地处理。

减少了I/O请求的排队等待时间

✓如果一个请求访问多个块，就可以由多个盘合作来并行处理。

提高了单个请求的数据传输率

3. 问题：阵列中磁盘数量的增加会导致磁盘阵列可靠性的下降。

如果使用了N个磁盘构成磁盘阵列，那么整个阵列的可靠性将降低为单个磁盘的 $1/N$ 。

✓解决方法：在磁盘阵列中设置冗余信息盘

当单个磁盘失效时，丢失的信息可以通过冗余盘中的信息重新构建。

8.3 廉价磁盘冗余阵列RAID

4. 廉价磁盘冗余阵列RAID

Redundant Arrays of Inexpensive Disks

✓ 独立磁盘冗余阵列

Redundant Arrays of Independent Disks

5. 大多数磁盘阵列的组成可以由以下两个特征来区分：

✓ 数据交叉存放的粒度

（可以是细粒度的，也可以是粗粒度的）

- **细粒度磁盘阵列**是在概念上把数据分割成相对较小的单位交叉存放。

8.3 廉价磁盘冗余阵列RAID

- **优点：**所有I/O请求都能够获得很高的数据传输率。
- **缺点：**在任何时间，都只有一个逻辑上的I/O在处理当中，而且所有的磁盘都会因为为每个请求进行定位而浪费时间。
- **粗粒度磁盘阵列**是把数据以相对较大的单位交叉存放。
 - 多个较小规模的请求可以同时得到处理。
 - 对于较大规模的请求又能获得较高的传输率。
- ✓ 冗余数据的计算方法以及在磁盘阵列中的存放方式

8.3 廉价磁盘冗余阵列RAID

6. 在磁盘阵列中设置冗余需要解决以下两个问题：

✓ 如何计算冗余信息？

- 大多都是采用奇偶校验码；
- 也有采用汉明码（Hamming code）或Reed-Solomon码的。

✓ 如何把冗余信息分布到磁盘阵列中的各个盘？

有两种方法：

- 把冗余信息集中存放在少数的几个盘中。
- 把冗余信息均匀地存放到所有的盘中。

（能避免出现热点问题）

8. 3 廉价磁盘冗余阵列RAID-RAID的分级及其特性

7. RAID的分级及其特性

RAID级别	可以容忍的故障个数以及当数据盘为8个时，所需要的检测盘的个数	优 点	缺 点	公司产品
0 非冗余，条带存放	0个故障； 0个检测盘	没有空间开销	没有纠错能力	广泛应用
1 镜像	1个故障； 8个检测盘	不需要计算奇偶校验，数据恢复快，读数据快。而且其小规模写操作比更高级别的RAID快	检测空间开销最大（即需要的检测盘最多）	EMC， HP（Tandem）， IBM
2 存储器式ECC	1个故障； 4个检测盘	不依靠故障盘进行自诊断	检测空间开销的级别是 $\log_2 m$ 级（m为数据盘的个数）	没有
3 位交叉奇偶校验	1个故障； 1个检测盘	检测空间开销小（即需要的检测盘少），大规模读写操作的带宽高	对小规模、随机的读写操作没有提供特别的支持	外存概念

8.3 廉价磁盘冗余阵列RAID-RAID的分级及其特性

RAID级别	可以容忍的故障个数 以及当数据盘为8个时， 所需要的检测盘的个数	优 点	缺 点	公司产品
4 块交叉奇偶校验	1个故障； 1个检测盘	检测空间开销小，小 规模的读操作带宽更 高	校验盘是小规模写的瓶颈	网络设备
5 块交叉分布 奇偶校验	1个故障； 1个检测盘	检测空间开销小，小 规模的读写操作带宽 更高	小规模写操作需要访问磁盘 4次	广泛应用
6 P+Q双奇偶校验	2个故障； 2个检测盘	具有容忍2个故障的 能力	小规模写操作需要访问磁盘 6次，检测空间开销加倍 (与RAID3、4、5比较)	网络设备

8.3 廉价磁盘冗余阵列RAID-有关RAID的几个问题

8. 有关RAID的几个问题

✓ 关键问题：如何发现磁盘的故障

- 在磁盘扇区中除了保存数据信息外，还保存有用于发现该扇区错误的检测信息。

✓ 设计的另一个问题

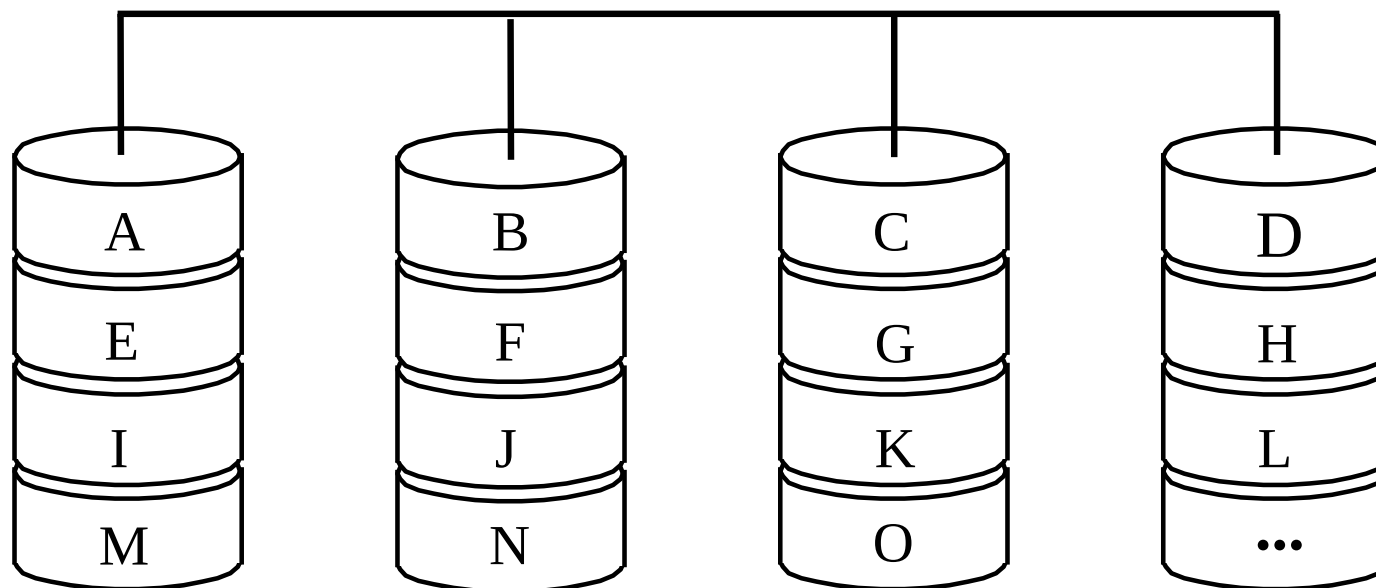
- 如何减少平均修复时间MTTR
- 典型的做法：在系统中增加热备份盘

✓ 热交换技术

- 与热备份盘相关的一种技术

8.3 廉价磁盘冗余阵列RAID-8.3.1 RAID0

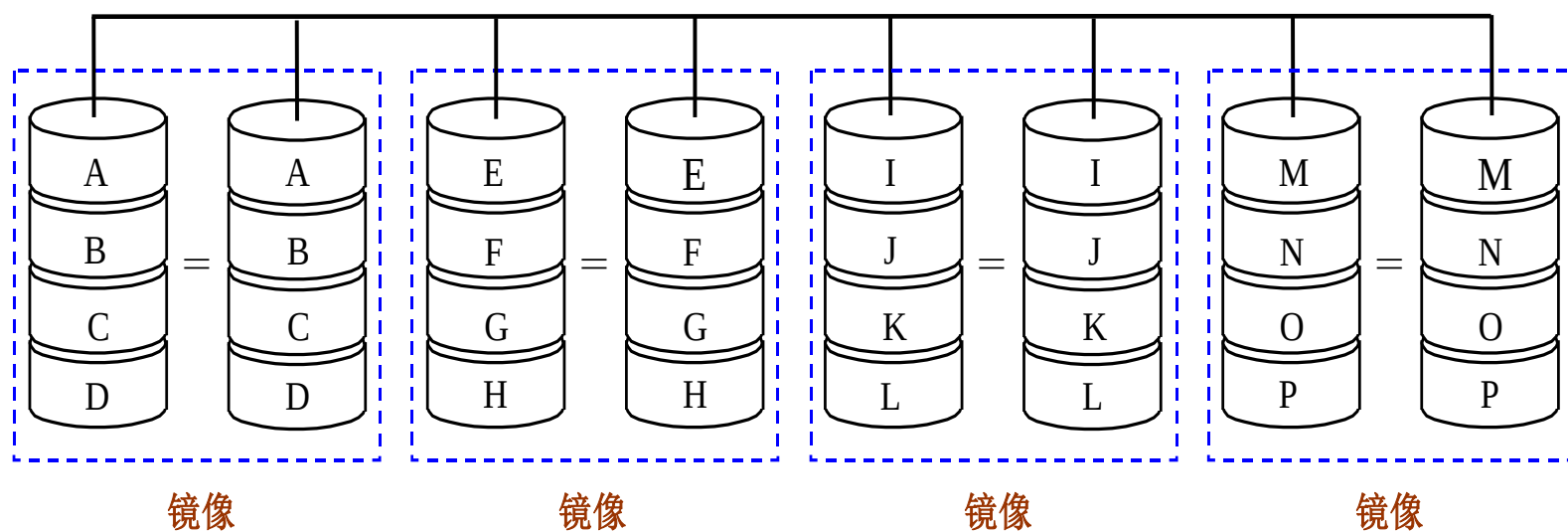
1. 非冗余磁盘阵列，无冗余信息。
2. 严格地说，它不属于RAID系列。
3. 把数据切分成条带，以条带为单位交叉地分布存放多个磁盘中。



8.3 廉价磁盘冗余阵列RAID-8.3.2 RAID1

1. 镜像磁盘，对所有的磁盘数据提供一份冗余的备份。

- ✓ 每当把数据写入磁盘时，将该数据也写入其镜像盘。在系统中所有的数据都有两份。



8.3 廉价磁盘冗余阵列RAID-8.3.2 RAID1

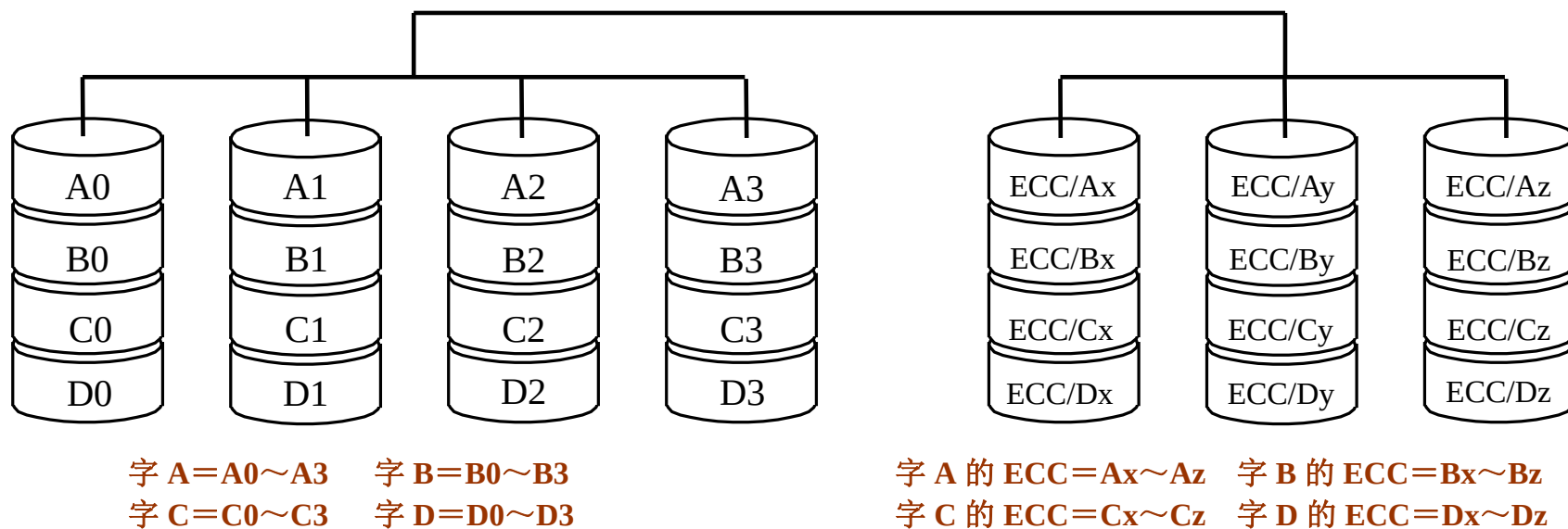
2. RAID1的特点

- ✓ 能实现快速的读取操作。
- ✓ 对于写入操作，镜像的两个磁盘都要写入。但可并行进行，而且不需要计算校验信息，所以其速度比级别更高的RAID都快。
- ✓ 可靠性很高，数据的恢复很简单。
- ✓ 实现成本最高。

8.3 廉价磁盘冗余阵列RAID-8.3.3 RAID2

1. 存储器式的磁盘阵列（按汉明纠错码的思路构建）

✓ 含4个数据盘的RAID2

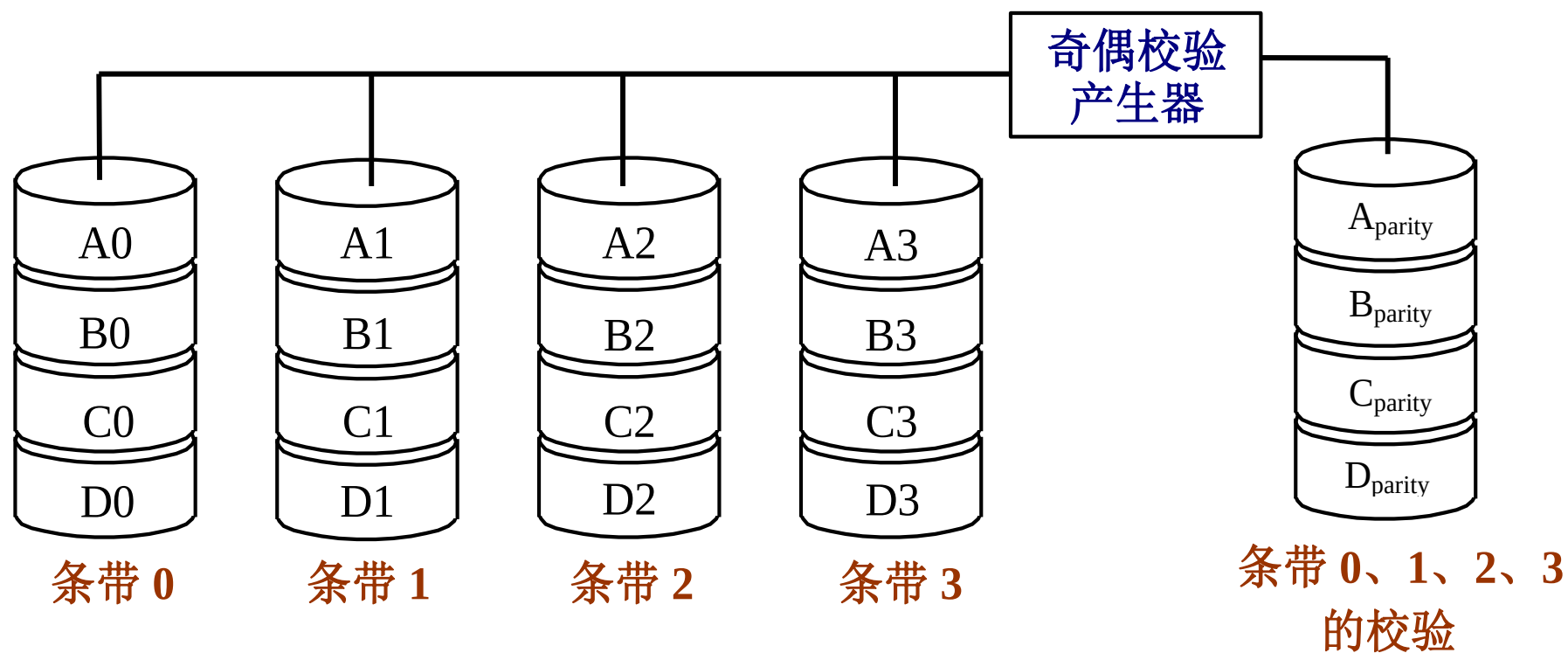


2. RAID2的特点

- ✓ 每个数据盘存放所有数据字的一位
(位交叉存放)
- ✓ 各个数据盘上的相应位计算汉明校验码，编码位被存放在多个校验(ECC)磁盘的对应位上。
- ✓ 冗余盘是用来存放汉明码的，其个数为 $\log_2 m$ 级。
 m : 数据盘的个数 (也就是数据字的位数)
- ✓ 并未被广泛应用，目前还没有商业化产品。

8.3 廉价磁盘冗余阵列RAID-8.3.4 RAID3

1. 位交叉奇偶校验磁盘阵列



2. RAID3的特点

✓ 采用奇偶校验

- 写数据时：为每行数据形成奇偶校验位并写入校验盘
- 读出数据时：如果控制器发现某个磁盘出故障，就可以根据故障盘以外的所有其他盘中的正确信息恢复故障盘中的数据。（通过异或运算实现）

✓ 细粒度的磁盘阵列，即采用的条带宽度较小。

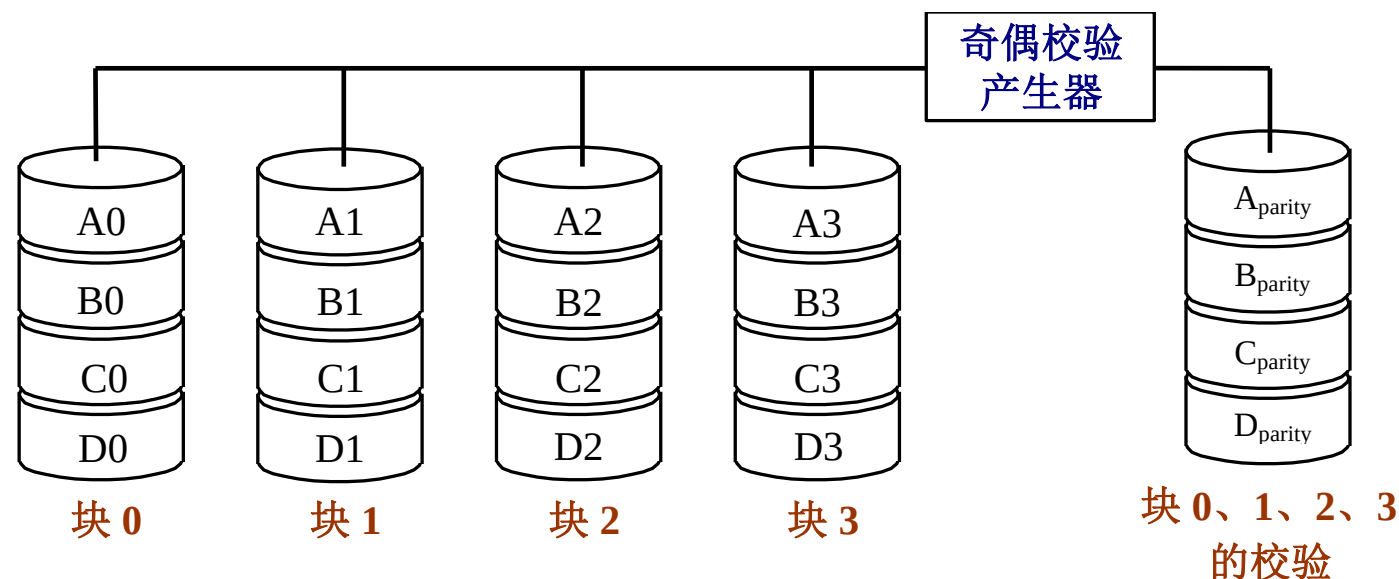
（可以是1个字节或1位）

- 能够获得很高的数据传输率，这种磁盘阵列对大数据量的读写具有很大的优越性。
- 不能同时进行多个I/O请求的处理。

✓ 只需要一个校验盘，校验空间开销比较小。

8.3 廉价磁盘冗余阵列RAID-8.3.5 RAID4

1. 块交叉奇偶校验磁盘阵列
2. 采用比较大的条带，以块为单位进行交叉存放和计算奇偶校验。
 - ✓ 实现目标：能同时处理多个小规模访问请求



3. RAID4读写特点

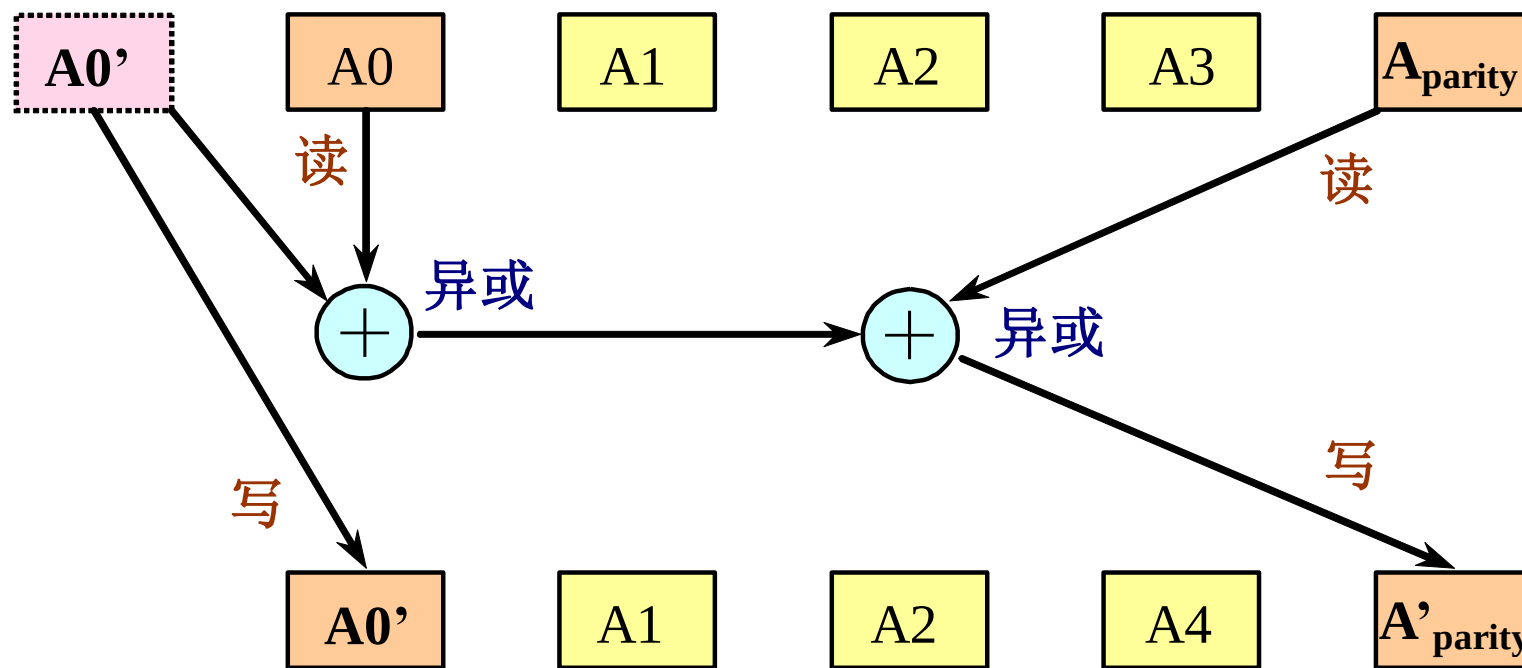
✓ 读取操作

- 每次只需访问数据所在的磁盘。
- 仅在该磁盘出现故障时，才会去读校验盘，并进行数据的重建。

✓ 写入操作

- 假定：有4个数据盘和一个冗余盘。
- 写数据需要2次磁盘读和2次磁盘写操作。

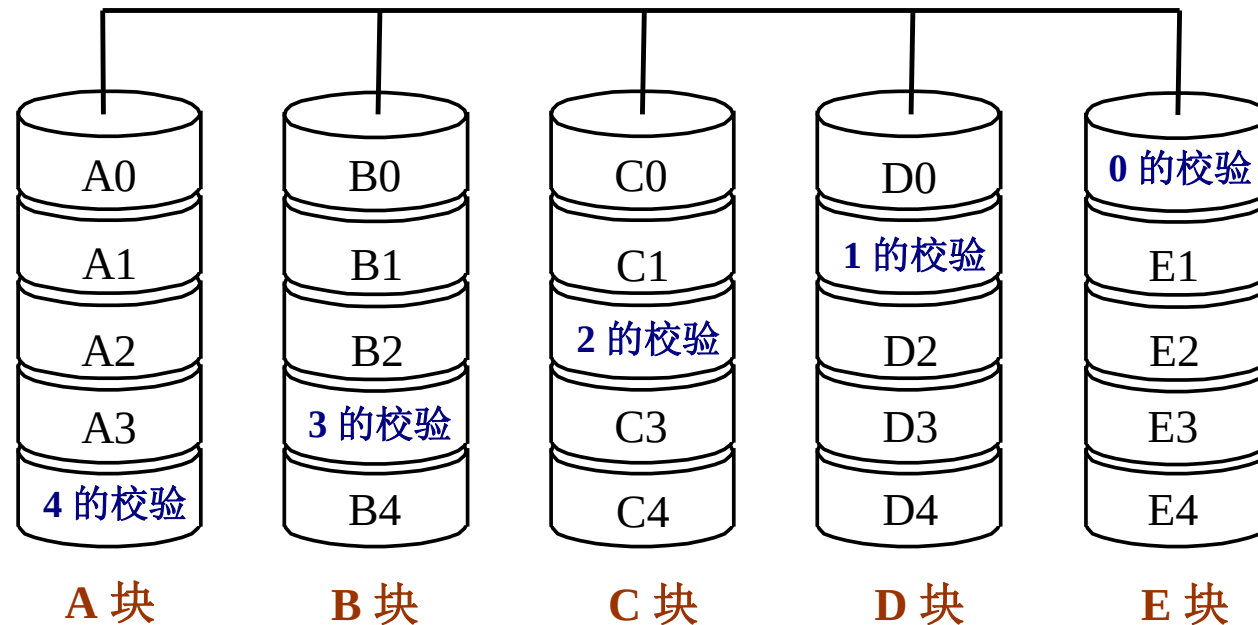
8.3 廉价磁盘冗余阵列RAID-8.3.5 RAID4



- ✓ RAID4能有效地处理小规模访问，快速处理大规模访问，校验空间开销比较小。但其控制比较复杂。

8.3 廉价磁盘冗余阵列RAID-8.3.6 RAID5

1. 块交叉分布奇偶校验磁盘阵列
2. 数据以块交叉的方式存于各盘，无专用冗余盘，奇偶校验信息均匀分布在所有磁盘上。

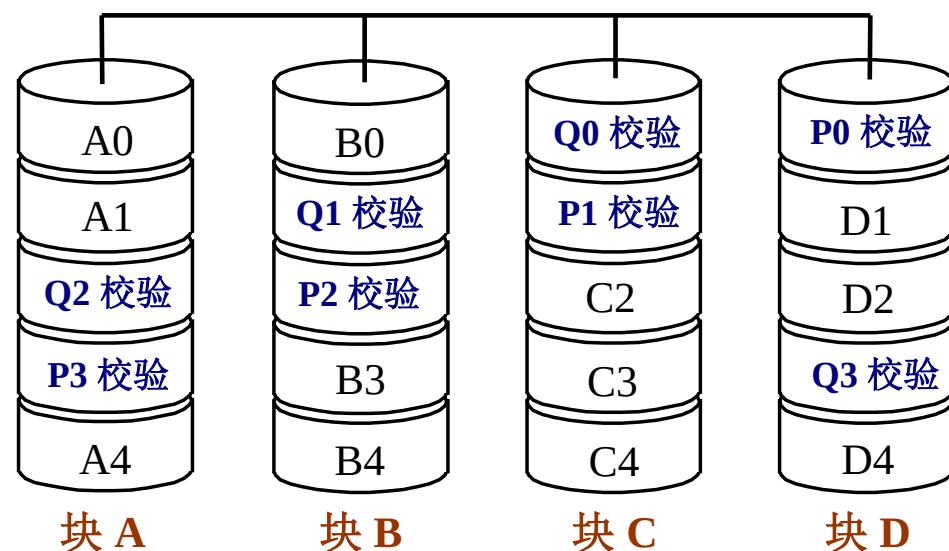


8.3 廉价磁盘冗余阵列RAID-8.3.7 RAID6

1. P+Q双校验磁盘阵列

2. 特点

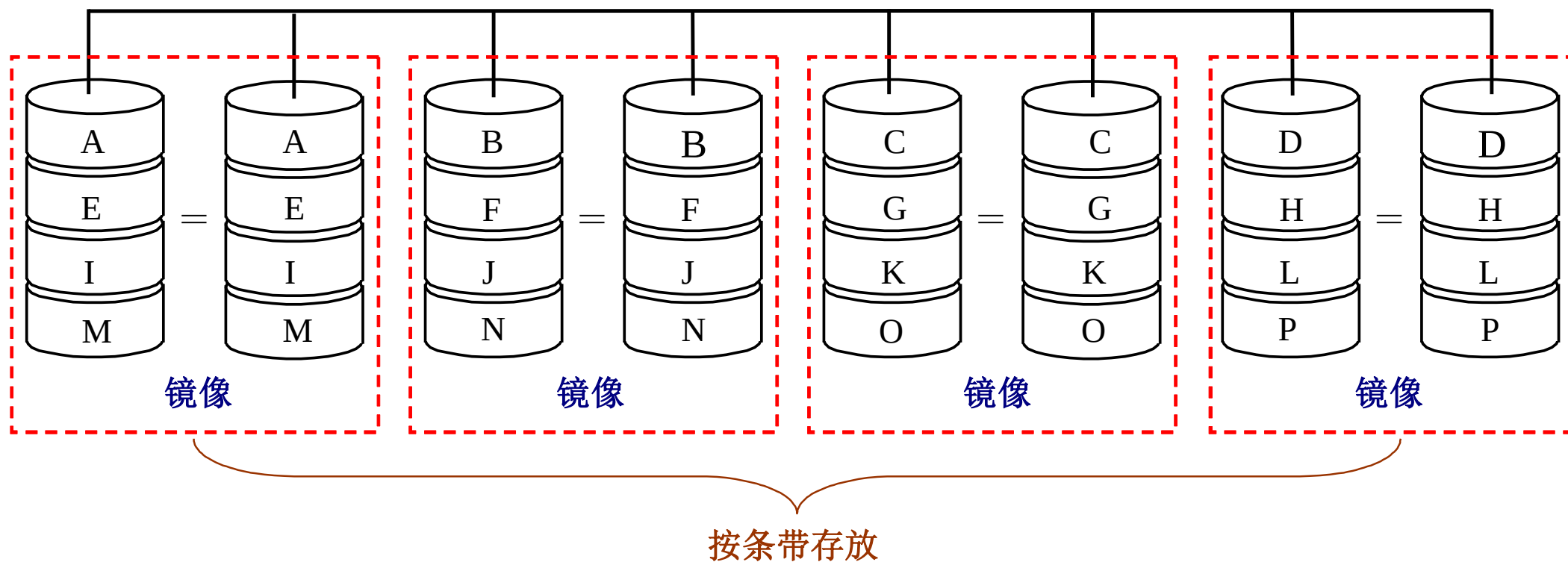
- ✓ 校验空间开销是RAID5的两倍
- ✓ 容忍两个磁盘出错



8.3 廉价磁盘冗余阵列RAID-8.3.8 RAID10与RAID01

1. RAID10又称为RAID1+0

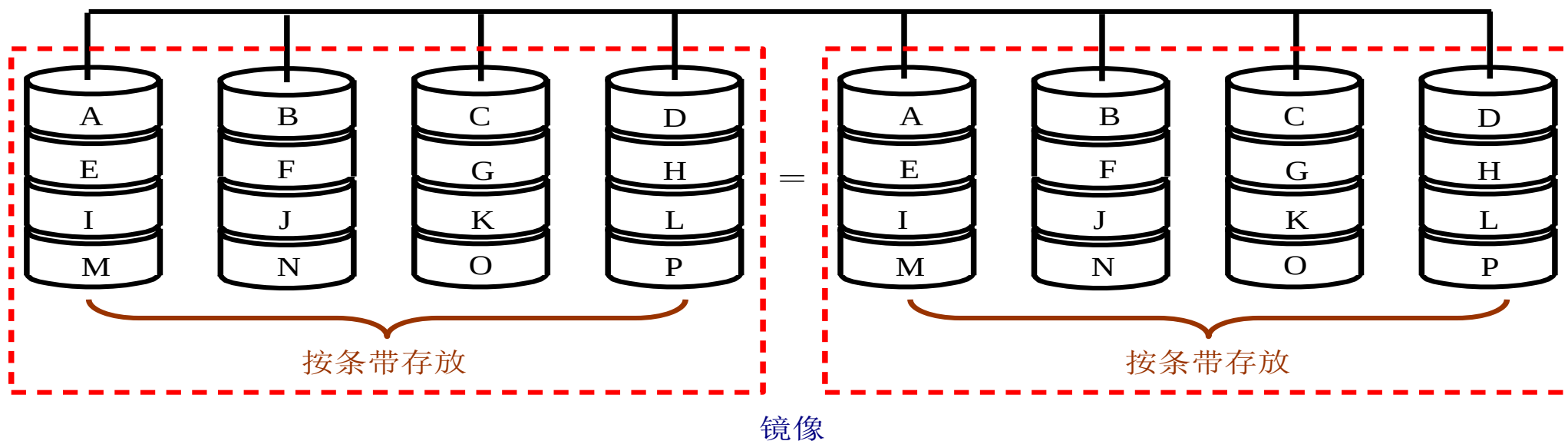
先进行镜像（RAID1），再进行条带存放（RAID0）



8.3 廉价磁盘冗余阵列RAID-8.3.8 RAID10与RAID01

2. RAID01又称为RAID0+1

先进行条带存放（RAID0），再进行镜像（RAID1）

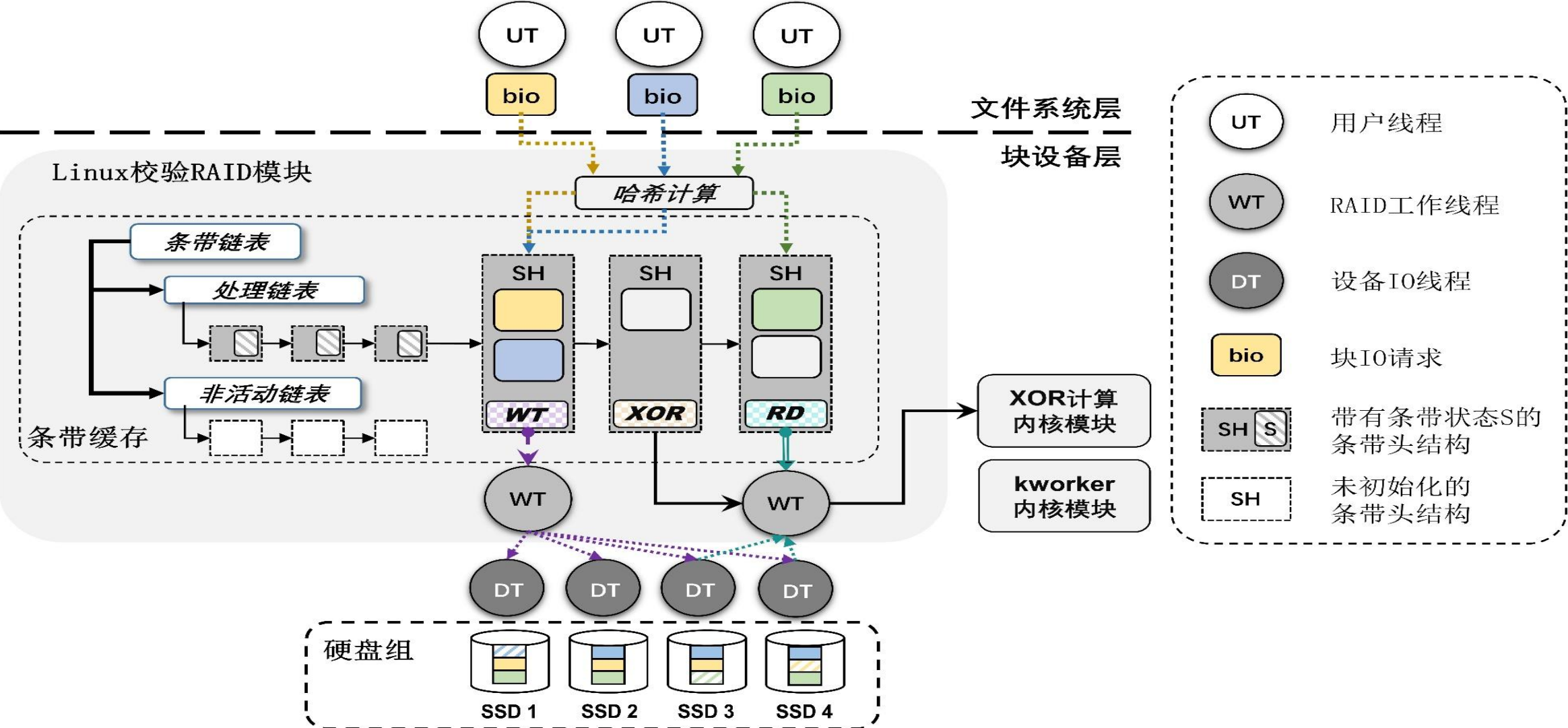


8.3 廉价磁盘冗余阵列RAID-8.3.9 RAID的实现与发展

1. 实现盘阵列的方式主要有三种：

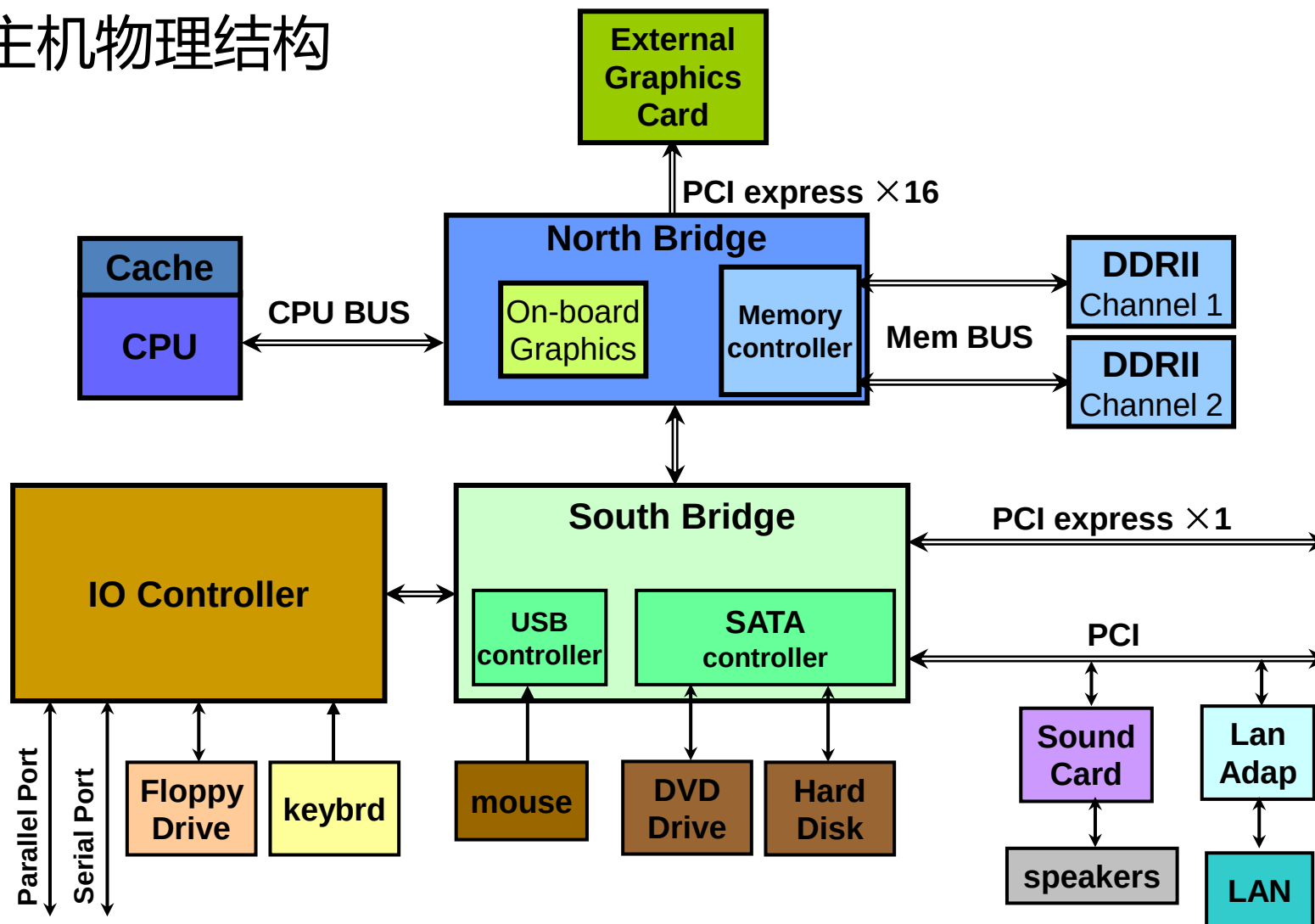
- ✓ 软件方式：阵列管理软件由主机来实现 Linux-MD
- ✓ 阵列卡方式：把RAID管理软件固化在I/O控制卡上，从而可不占用主机时间，一般用于工作站和PC机。
- ✓ 子系统方式：一种基于通用接口总线的开放式平台，可用于各种主机平台和网络系统。
 - 低端、中端和高端（EMC）

8.3 廉价磁盘冗余阵列RAID-8.3.9 RAID的实现与发展-Linux MD



6.5 存储系统结构-6.5.1存储处理架构-物理结构

■ 一种典型主机物理结构



6.5 存储系统结构-6.5.1存储处理架构- CPU对I/O设备的编址方式

CPU对I/O设备的编址有两种方式

□ 存储器映射I/O（也称为I/O设备统一编址）

- ◆ 将一部分存储器地址空间分配给I/O设备，用load指令和store指令对这些地址进行读写将引起I/O设备的数据传输。
- ◆ 将一部分存储空间留出用于设备控制，对这一部分地址空间进行读写就是向设备发出控制命令。

□ 给I/O设备独立编址

- ◆ 需要在CPU中设置专用的I/O指令来访问I/O设备。
- ◆ CPU需要发出一个标志信号来表示所访问的地址是I/O设备的地址。

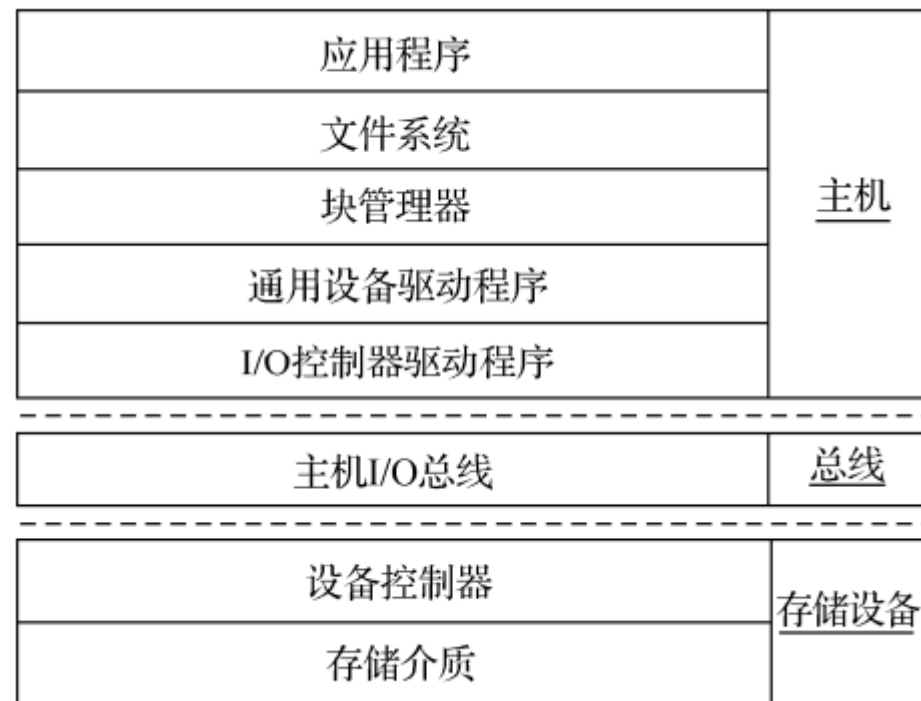
6.5 存储系统结构-6.5.1 存储处理架构

- 存储系统不仅需要高性能I/O和可靠大容量存储，还需要提供数据存储组织管理的能力
- 应用程序可以通过标准接口方便存取海量数据
- 存储系统硬件包含三个部分，主机、总线和存储设备
 - 主机端存储功能进一步分为应用程序、文件系统、块存储模块、通用块设备驱动程序和I/O驱动模块
 - I/O驱动模块发送I/O请求通过总线达到设备端，设备控制器解析I/O请求，转换为对于物理存储介质进行读写操作
- 存储栈（Storage Stack）
 - 应用数据请求逐层下发送到设备，之后再逐层向上响应请求。每层都执行特定的处理
- 存储栈中每一层都会逻辑存取下一层中的“数据”
 - 这意味着，每一存储层需要给上层提供给存储视图（存储空间），类似于指令集体系结构中虚拟内存编址和存取
- 本地数据存储过程可以抽象为“在三级存储空间执行两次映射”
 - 第一级是文件命名空间到逻辑存储空间（数据块/页空间）的映射
 - 第二级是逻辑存储空间（卷/单元）到物理存储空间（存储设备/分区/单元）的映射
 - 第一级映射需要构建额外索引维护每个文件映射，索引自身也需存储。第二级映射通常采用地址算术映射方式

6.5.1 存储处理架构 - 存储栈

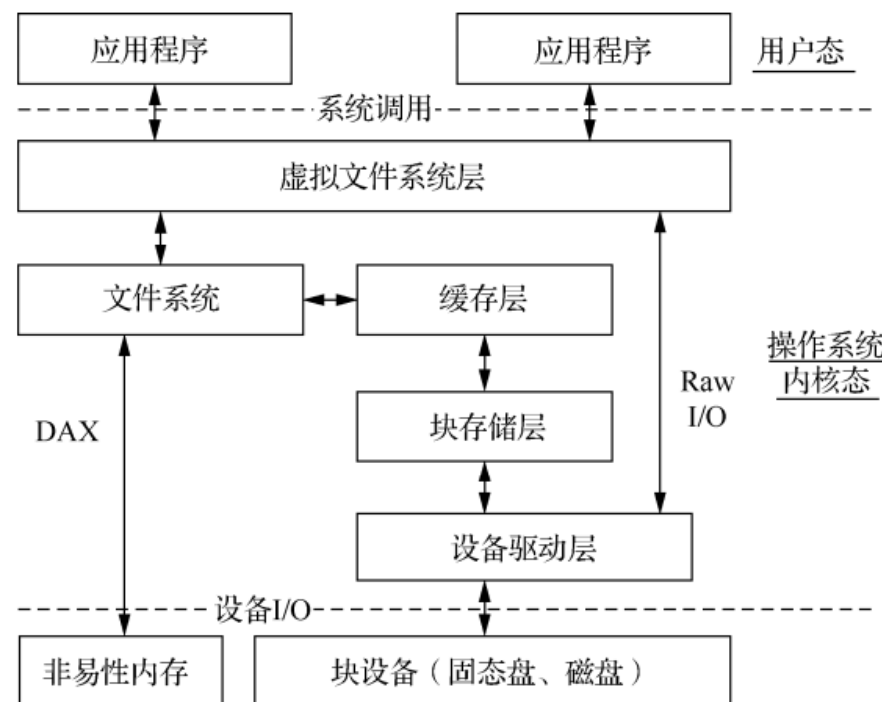
■ 存储栈处理过程

1. 文件系统为用户和应用程序提供文件目录树视图
内部把文件/目录读写请求转化为逻辑块层请求
2. 而逻辑块层向上提供统一数据块地址空间，内部把逻辑块请求转换为设备块I/O请求
3. 通用块层也提供I/O合并、写吸收等I/O优化调度之后发送到相应的设备驱动层
4. 设备驱动层执行设备相关的优化，例如针对磁盘I/O调度方法对于I/O请求重排序，减少磁盘来回寻道
5. I/O设备驱动程序执行物理I/O，通过总线发送给设备端的控制器
6. 而设备控制器具体负责对其内部存储介质的数据读写



6.5.2 本地存储-数据处理流程

- Linux本地存储系统结构，包括应用程序、操作系统和存储设备之间的数据处理流程
- 应用程序通过用户态标准I/O库，调用标准文件接口存取文件数据
- 文件系统把文件请求转化为相应数据块读写操作，通过缓存层、块存储层和驱动层，最终交给存储设备
 - 用户态标准I/O库会发起系统调用进入操作系统内核态，进而调用相应虚拟文件系统接口函数
 - 虚拟文件系统层提供通用文件存取接口，可以同时接受多个文件操作请求，进而调用具体文件系统实例的文件系统处理函数，虚拟文件系统层可以构建在多个文件系统实例之上
 - 文件系统把文件请求转化为相应数据块存取
 - 为了提高性能，操作系统提供页缓存机制加速数据块存取
 - 仅在数据块读缺失、或者脏块写入存储设备时，把相应块请求发送到通用块I/O层，这种内核处理模式保证文件系统实例是全局共享的，能够为多个应用程序提供并发文件操作服务



6.5.2 本地存储- 设备驱动程序

- 现代操作系统为每个设备提供设备驱动程序，不同设备具有不同内部结构和操作时序
- 驱动程序定义一组编程接口
 - 完全隐藏设备的工作细节，内部把这些调用映射到物理设备的特有I/O操作上，包括对设备进行初始化、参数有效性检查、执行I/O过程、管理电源和日志等
 - 其他程序通过这组标准化调用实现高层设备存取，从而独立于特定设备的驱动程序。这也是计算机分层设计原则的应用
- 设备驱动程序会检查设备以确定请求是否能够被处理，如是则把相应的I/O指令发送给设备，在大多数情况下会等待直到设备完成该指令
 - 在阻塞模式下，驱动程序可以通过中断来解除阻塞状态
 - 在非阻塞模式下，驱动程序可以通过轮询检测完成信号，尽快响应请求完成
 - 设备驱动程序必须是可重入的，因为设备驱动程序会阻塞和唤醒然后再次阻塞。驱动程序不允许进行系统调用，但是它们通常需要与内核的其余模块进行交互
- 存储设备的I/O过程在主机和设备之间传输数据块，其I/O请求语义比较简单，通常包含读/写、设备ID、请求起始地址和请求数据块数量

6.5.2 本地存储-文件系统-1 文件系统整体功能

- 文件系统和存储设备互为表里。存储设备是文件系统的底层物理“经济基础”，文件系统是存储设备的逻辑“上层建筑”
- 对应用程序而言，文件系统是最重要数据管理和存储系统
 - 对于文件和目录操作属于高层存取语义，方便应用程序的使用
 - 而数据块接口为低层简单块存取语义
- 文件系统为应用提供文件/目录数据抽象及相应处理接口，内部维护文件逻辑空间到物理存储空间之间的映射，并执行数据处理操作，屏蔽底层存储特性和I/O过程

6.5.2 本地存储-文件系统-2文件目录概念

- 存储系统中每个数据对象需要有唯一的名字（或者ID）
 - 应用程序才能通过名字编程访问数据对象，因此也可以认为逻辑存储空间就是存储对象的名字空间
- 文件是一个抽象的数据容器（数据对象），具有独立逻辑地址空间
 - 应用程序可以把数据序列化后保存到文件之中
 - 不同类型文件具有特定的内部数据组织格式（数据结构），例如标准视频和音频文件等
- 文件系统以文件和目录结构形式对外提供的统一命名空间，文件目录存储空间也称为命名空间，通常是树形结构的
 - 文件目录抽象容易被用户理解，容易扩展，方便寻址，有利于大量文件的组织和管理
- 文件命名空间一般会有根目录和路径
 - 绝对路径和相对路径

6.5.2 本地存储-文件系统-3文件目录结构和操作

- 文件存取操作，IEEE制定的POSIX（Portable Operating System Interface）是应用程序访问文件系统的API标准，定义了文件级存取接口及其操作语义

文件操作类型	文件/目录操作接口
文件系统	mount, unmount, interrupt
元数据操作	LOOKUP, FORGET, BATCH FORGET, CREATE, UNLINK, LINK, RENAME, RENAME2, OPEN, RELEASE, STATFS,FSYNC, FLUSH, ACCESS
数据操作	READ, WRITE
属性操作	GETATTR, SETATTR
扩展属性操作	SETXATTR, GETXATTR, LISTXATTR, REMOVEXATTR
符号链接	SYMLINK, READLINK
目录操作	MKDIR, RMDIR, OPENDIR, RELEASEDIR, READDIR, READDIRPLUS, FSYNCDIR
锁操作	GETLK, SETLK, SETLKW
其他	BMAP, FALLOCATE, MKNOD, IOCTL, POLL, NOTIFY REPLY

6.5.2 本地存储-文件系统-4块存储空间

- 块存储空间：由一组固定大小单元线性编址构成，固定单元通常称为数据块Block（有时也称为页、段等），每个块具有唯一的地址编号Block-ID，存储空间大小就是数据块数量Block-Num和块大小Block-Size的乘积，存取块地址空间的数据比较简单，可以通过数据地址计算得到所属块地址和块内偏移
- 三个地方提供块存储空间：文件内部、逻辑卷、存储卷
- CPU访问的内存空间一般都是按照字节编址，其指令直接寻址范围显然受到CPU字长的限制
- CPU访问外储系统时，可以使用I/O请求中的间接寻址扩充可访问的外存储空间。数据块的逻辑地址称为LBA（Logical Block Address），而存储设备上的具体磁盘物理扇区或者固态盘的物理页数称之为物理块地址PBA（Physical Block Address）

6.5.2 本地存储-文件系统-5文件/块地址映射

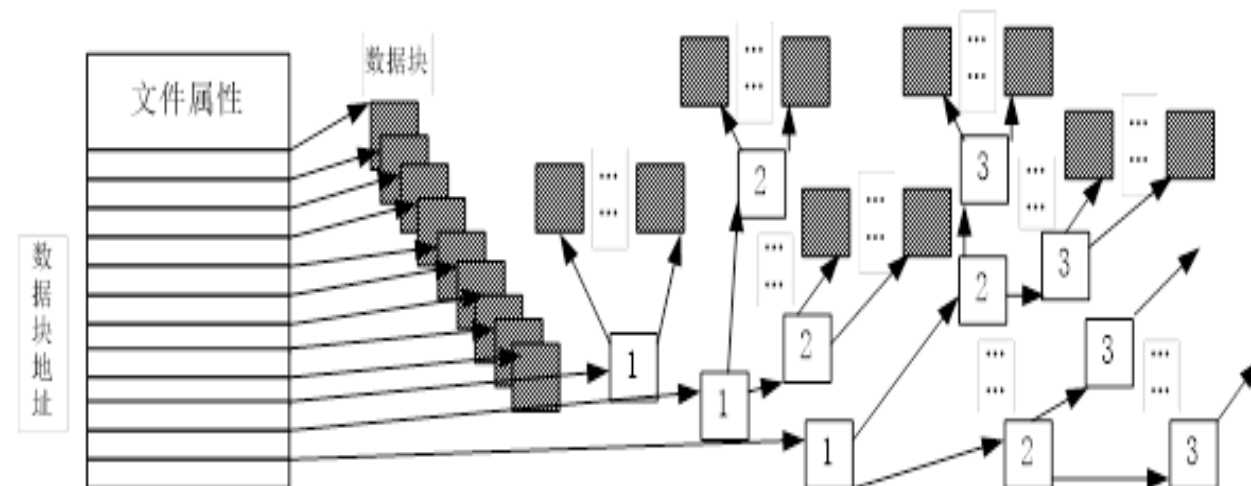
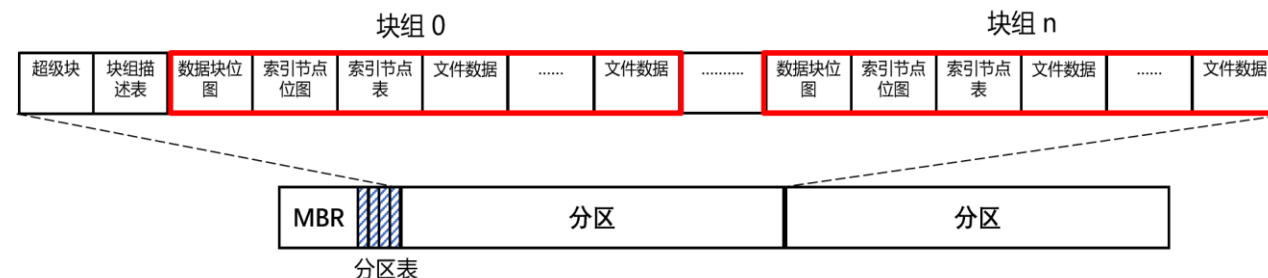
■ 文件系统实现文件和文件目录结构到逻辑卷空间之间的映射

□ 核心是一组数据结构，称之为元数据

■ 文件系统的元数据

□ 超级块、存储分配表、文件索引表、文件索引区和数据区等盘上数据结构
(On-disk Data Structure)

□ inode索引结构除记录文件的属性之外，inode中最重要的数据结构就是实现文件块-数据块索引结构，也就是文件内部逻辑块地址到逻辑卷块地址映射



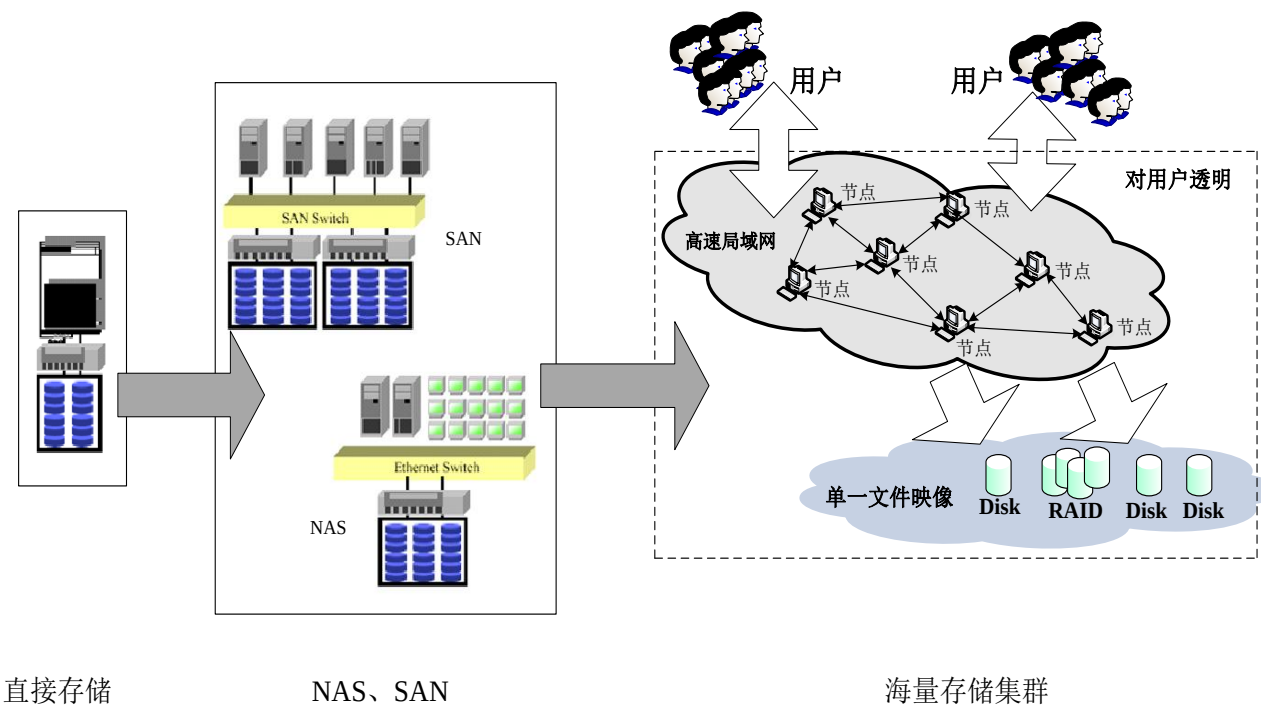
6.5.3 网络存储系统 -1本地和网络存储

■ 本地存储结构称之为DAS结构 (Direct-Attached Storage)

- 在单机系统中，CPU通过外部（可能是多级）总线连接到物理存储设备，操作系统管理和组织存储设备中的数据，并且通过存储栈存取
- 受限于总线电气特性和带宽，不能连接过多的存储设备

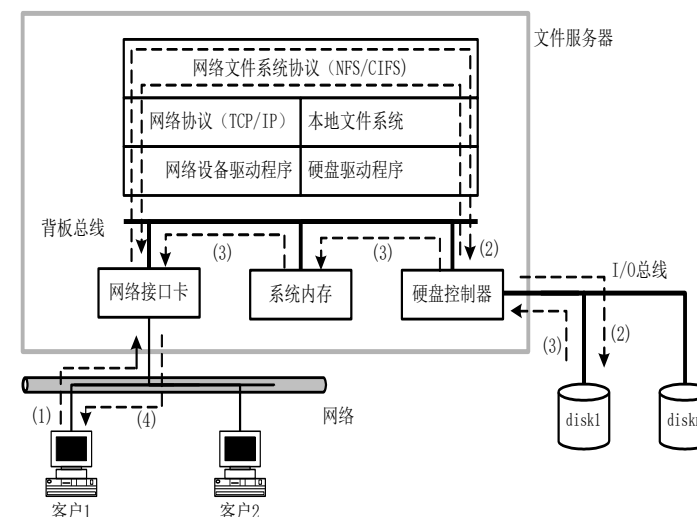
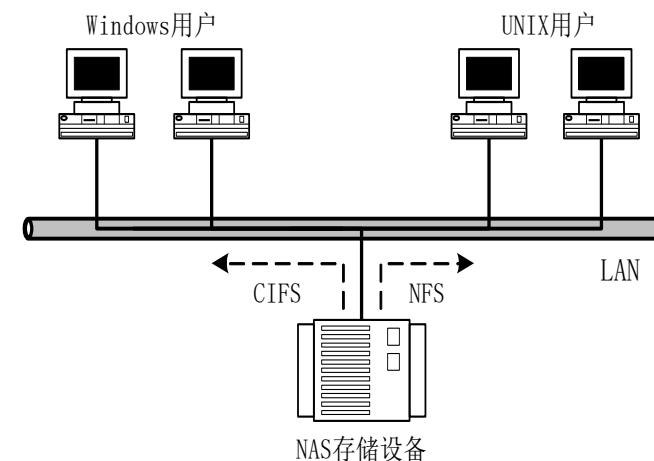
■ 四种基本网络存储结构

- 存储区域网 (SAN)
- 附网存储 (NAS)
- iSCSI存储
- 分布式文件系统



6.5.3 网络存储系统 -2 附网存储 (NAS)

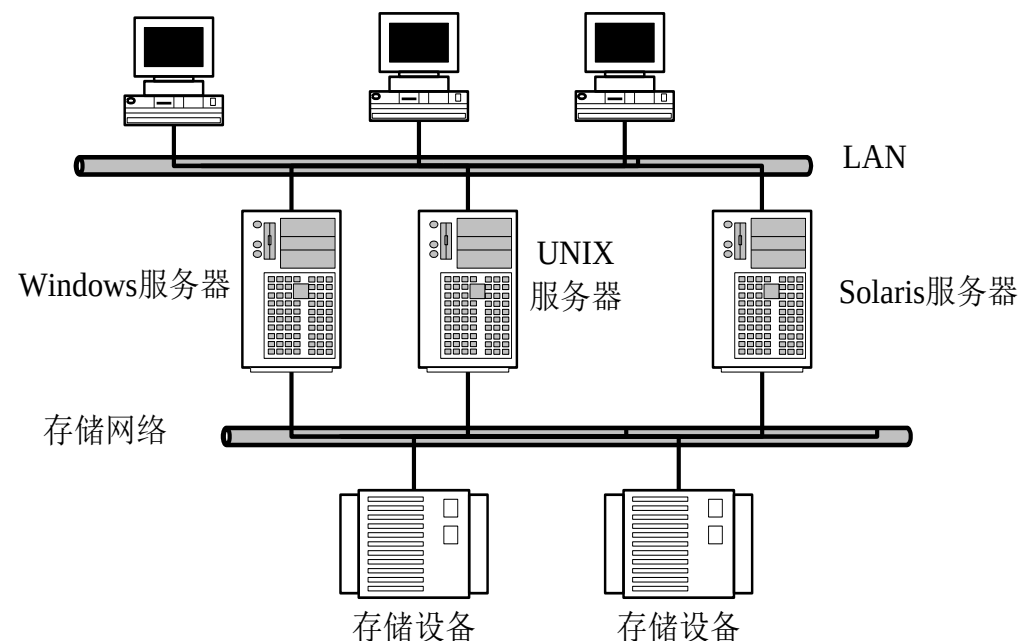
- NAS 是通过网络向用户提供文件级服务的存储服务器，它将硬件和软件有机地集成在一起，为多个用户提供文件视图
 - (1) 异构平台下的文件共享：不同操作系统平台下的多个客户端可以很容易地共享NAS中的同一个文件
 - (2) 充分利用现有的局域网 (LAN) 结构
 - (3) 容易安装，使用和管理都很方便，实现即插即用
 - (4) 广泛的适用性：由于基于IP/Ethernet以及标准的NFS和CIFS，可以适应复杂的网络环境
 - (5) 较低的总拥有成本



6.5.3 网络存储系统 -3 存储区域网 (SAN)

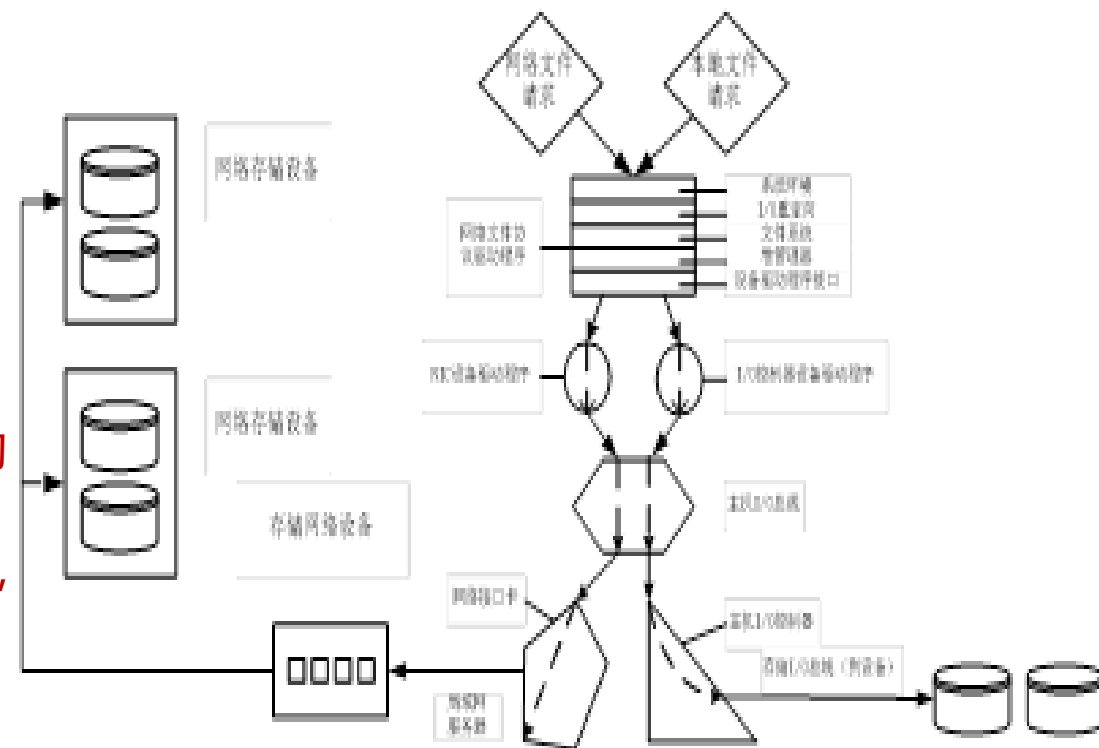
- SAN是一种以网络为中心的存储结构。按照SNIA定义，SAN是一种利用专用网络 (Fibre Channel) 连接起来的可以在服务器和存储设备之间直接传送数据的存储网络系统。SAN可以理解对于本地总线扩展，变成交换结构。存储设备和SAN中的应用服务器之间采用的是block I/O的方式进行数据交换

(1) 高性能、高速存取：目前Infiniband可提供100Gbps的带宽；(2) 高可用性：具有更多的存储空间进行冗余；(3) 集中存储和管理：通过整合各种不同的存储设备形成一个统一的存储池；(4) 高可扩展性：服务器和存储设备相分离，两者的扩展可以独立进行；(5) 支持更远的距离，另外通过光纤通道网卡、集线器、交换机等互联设备，用户可根据需要可以灵活地放置服务器和存储设备。在数据库等其它面向事务处理的应用场合，SAN的优势更加明显。SAN也有弱点：(1) 设备的互操作性较差：依赖于专用网络协议。(2) 多主机共享SAN需要额外协调软件



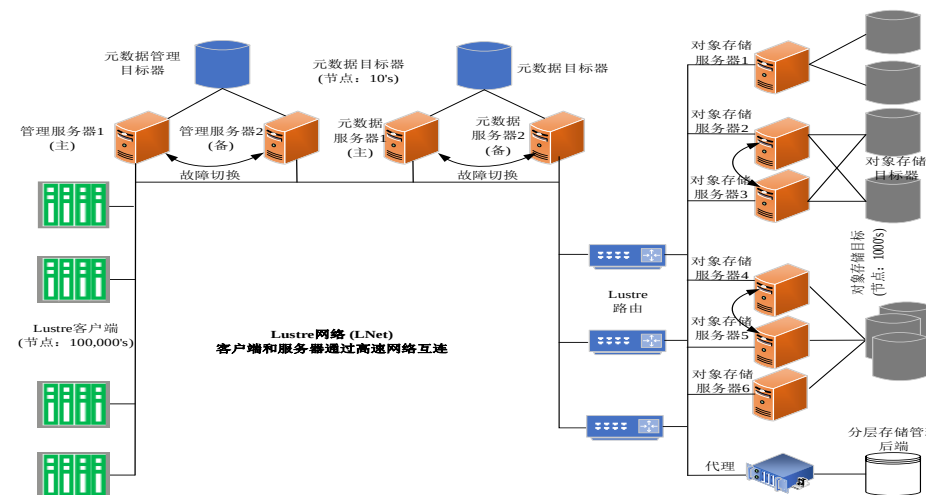
6.5.3 网络存储系统 -4 iSCSI存储

- iSCSI在TCP/IP封装SCSI存储协议，是一种IP的SAN
- iSCSI采用Initiator/Target (启动设备/目标设备)的实现模式：Initiator发出iSCSI命令，而 Target 接收到该命令后，负责解释和执行
 - (1) 当用户向一个iSCSI target设备发出访问请求时，应用程序通过系统调用访问文件系统
 - (2) 文件系统首先解释这个I/O命令，分析I/O命令所在的设备和地址。然后它把这个请求发送给SCSI上层驱动程序
 - (3) SCSI上层驱动程序把相应的I/O命令转化为SCSI命令，并发给iSCSI Initiator
 - (4) iSCSI Initiator把SCSI命令封装打包，通过网络发送到iSCSI Target模块
 - (5) 当iSCSI Target模块收到iSCSI命令包后，它把iSCSI命令包中的SCSI命令发送到SCSI的底层驱动程序
 - (6) 由SCSI底层驱动程序完成I/O请求。把数据按原路返回给用户

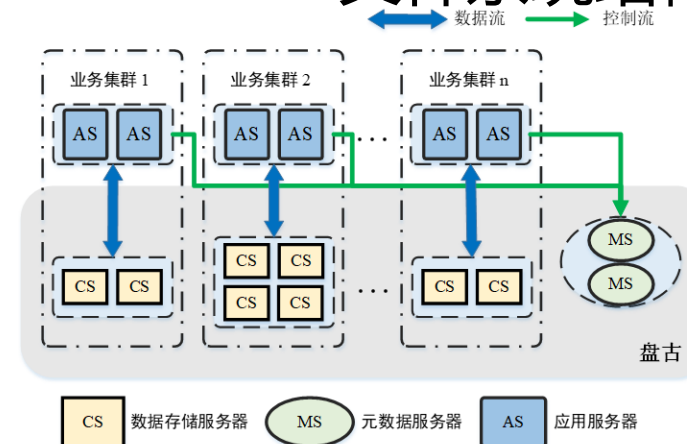


6.5.3 网络存储系统 -5 存储集群

- 存储集群把多个存储节点、元数据服务器和客户端服务器通过高速网络连接起来，通过元数据服务器对外提供统一命名空间。客户端从元数据服务器获取所请求文件的存储节点信息，然后直接存取相应的存储节点
- 这种三方合作I/O模式，使得存储集群具有良好的扩展性，能够扩展包含更多的存储节点和元数据服务器
- 存储集群通常使用分布式文件系统来统一管理所有存储节点（客户端、元数据服务器和存储节点）



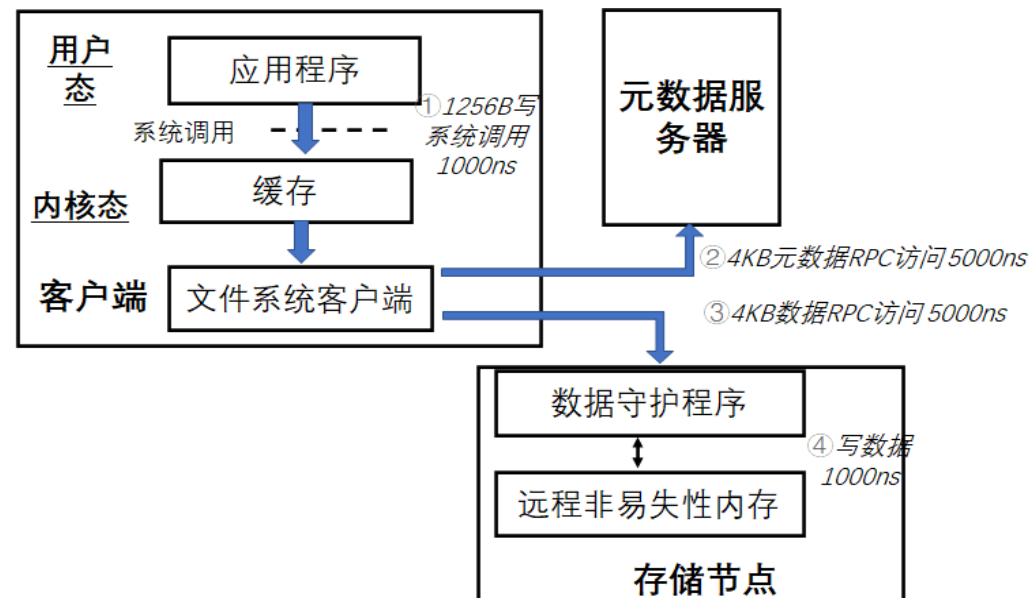
Lustre文件系统架构图



阿里盘古存储系统框架图

6.5.4 存储I/O路径分析

- 分析数据存取性能涉及到许多具体的细节，一个最为重要的方法就是分析数据的存取路径
- 存取路径包括物理过程和逻辑过程，前者是数据在硬件部件上实际数据传输的过程，而后者是软件对于数据的处理过程
- 存取路径是指请求和数据在存储系统中传递的通道
- 而存取路径就是从数据请求到物理存储设备之间的交互通道。之所以称之为交互通道，其原因在于它不是单向的，而是双向的，无论是读还是写过程都需要数据和命令的传输，这两个方面的过程往往占据双向过程中的某一边



8.6 I/O与操作系统-8.6.2 I/O和Cache数据一致性

1. Cache会使一个数据出现两个副本：

- ✓ 一个在Cache中，另一个在主存中。

2. I/O设备可以修改存储器中的内容

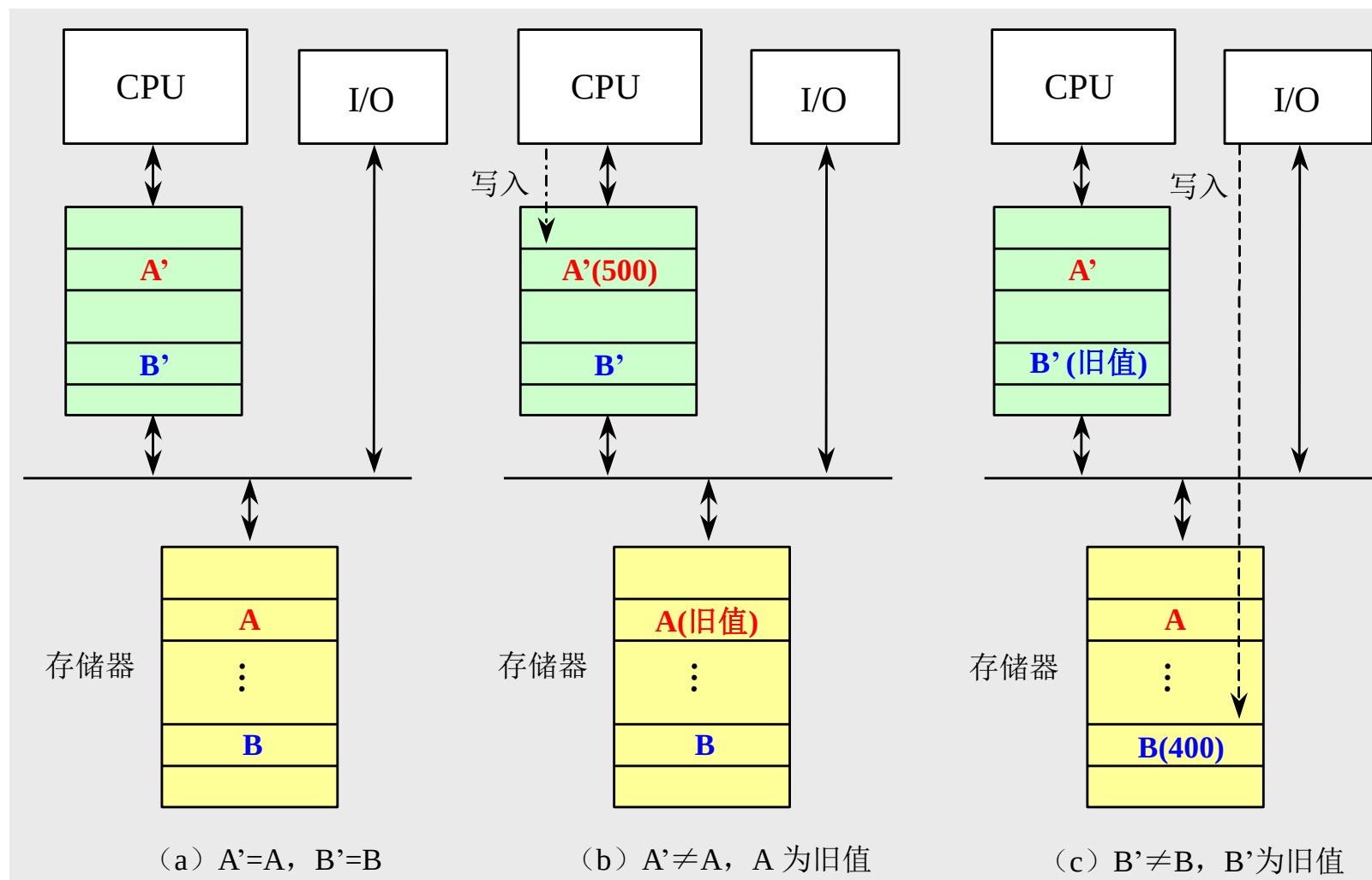
- ✓ 把I/O连接到存储器上

会出现以下情况：

- CPU修改了Cache的内容后，由于存储器的内容跟不上Cache内容的变化，I/O系统进行输出操作时所看到的数据是旧值。（写直达Cache没有这样的问题）
- I/O系统进行输入操作后，存储器的内容发生了变化，但CPU在Cache中所看到的内容依然是旧值。

8.6 I/O与操作系统-8.6.2 I/O和Cache数据一致性

举例：假设Cache采用写回法，并且A' 是A的副本，B' 是B的副本。



✓ 把I/O直接连接到Cache上

- 不会产生由I/O导致的数据不一致的问题。
 - 所有I/O设备和CPU都能在Cache中看到最新的数据。
- I/O会跟CPU竞争访问Cache，在进行I/O时，会造成CPU的停顿。
- I/O还可能会破坏Cache中CPU访问的内容，因为I/O操作可能导致一些新数据被加入Cache，而这些新数据可能在近期内并不会被CPU访问。

3. 解决内容一致性的方法

(不管Cache是采用写直达法还是写回法)

✓ 软件的方法

- 设法保证I/O缓冲器中的所有各块都不在Cache中。
- 具体做法有两种
 - 把I/O缓冲器的页面设置为不可进入Cache的，在进行输入操作时，操作系统总是把输入的数据放到该页面上。
 - 在进行输入操作之前，操作系统先把Cache中与I/O缓冲器相关的数据“赶出”Cache，即把相应的数据块设置为“无效”状态。

✓ 硬件的方法

- 在进行输入操作时，检查相应的I/O地址（I/O缓冲器中的单元）是否在Cache中（即是否有数据副本）。
- 如果发现I/O地址在Cache中有匹配的项，就把相应的Cache块设置为“无效”。

习题8.12混联系统可靠度

假定某网络型RAID系统包含6个SCSI磁盘，采用RAID 1+0结构，对给定时间 t ，各部分可靠度为：网络接口通道NIC的 $R_1=0.9$ ，阵列控制器 $R_2=0.95$ ，SCSI通道适配器 $R_3=0.95$ ，磁盘 $R_4=0.8$ 。

- (1) 画出系统可靠性框图；
- (2) 写出系统可靠性 R 的表达式，计算 R 的数值；
- (3) 提出进一步增强系统可靠性的若干建议。

